Лабораторна робота №7

Тема роботи: Аудит продуктивності вебзастосунку.

Мета роботи: навчитися знаходити причини повільної роботи вебзастосунку **вимірюванням**, а не здогадками. Студент має виправити знайдене й **довести** результат цифрами та трейсами з DevTools, React Profiler і Lighthouse.

Що ви отримуєте: готовий проєкт «Каталог репетиторів» (React 18 + Vite). Це SPA з пошуком, фільтрами, сортуванням, обраними й карткою з деталями. Дані (~2000 репетиторів) генеруються локально, тому у всіх однакові, а інтернет не потрібен. Застосунок працює правильно, але повільно. Інструкція із запуску — у `README.md` проєкту.

Хід роботи

Завдання 1. Базова лінія. Запустіть production-збірку (`npm run build` + `npm run preview`) і оберіть щонайменше 3 користувацькі сценарії: завантаження сторінки, введення тексту в пошук, дії зі списком (кліки, скрол). Для кожного сценарію зафіксуйте метрики «до». Це можуть бути LCP, CLS, INP або тривалість взаємодії, сума long tasks, обсяг завантажених даних. Кожну цифру візьміть як медіану з щонайменше 3 запусків і вкажіть умови: браузер і версію, CPU throttling, network throttling, режим інкогніто.

Завдання 2. Пошук причин. За допомогою DevTools (Performance, Network), React Profiler і Lighthouse знайдіть, чому застосунок повільний. У проєкті є **кілька вузьких місць різної природи**. Є також код, який лише виглядає повільним. Для кожної знайденої проблеми сформулюйте гіпотезу і підтвердіть її трейсом чи звітом: де витрачається час і чому.

Завдання 3. Виправлення й повторне вимірювання. Виправляйте свої знахідки по одній проблемі за раз, окремим комітом на кожну. Після кожної зміни переміряйте той самий сценарій у тих самих умовах. Функціональність застосунку має лишитися такою самою: пошук, фільтри, сортування, обрані, картка. Видаляти чи спрощувати функції заради швидкості не можна.

Завдання 4. Regression-тест. Для одного з виправлень напишіть автоматичний тест (Vitest + Testing Library, у `src/__tests__/`). Він має **падати на початковому коді й проходити після виправлення**. Тест перевіряє поведінку, яка була зламана, наприклад кількість рендерів, кількість викликів дорогої функції або наявність потрібних атрибутів. Він не має перевіряти назви змінних чи деталі реалізації. Усі існуючі тести мають проходити.

Завдання 5. Співбесіда (5 хв). На співбесіді ви демонструєте екран. Викладач називає одне з ваших виправлень, а ви знімаєте трейс наживо, показуєте в ньому, де була проблема, і пояснюєте, чому зміна її прибрала. Потім 1–2 додаткові питання про ваш код і методику.

Обов'язкові умови

1. Підсумкові цифри знімаються на **production-збірці** (`npm run preview`). Dev-режим можна використовувати лише для React Profiler.
2. Кожне виправлення має доказ у вигляді цифр до і після та скріншотів трейсу чи звіту до і після в однаковому масштабі. Зміна без вимірювання не зараховується, навіть якщо вона правильна.
3. Причина кожної проблеми пояснена 1–2 реченнями своїми словами: що саме відбувалося і чому це повільно.
4. Функціональність збережена, `npm test` проходить.

Формат здачі (Moodle):

- Посилання на репозиторій (GitHub/GitLab) або zip **без** **`node_modules`**. Історія комітів має починатися з початкового коду, далі окремий коміт на кожне виправлення.
- Файл `REPORT.md` у корені проєкту:
  - умови вимірювання (машина, браузер, throttling);
  - таблиця базової лінії;
  - для кожного виправлення: сценарій, метрика до і після, причина, що змінено, посилання на коміт;
  - список місць, які ви перевірили й **свідомо не чіпали**, з цифрою, чому це не проблема.
- Тека `report/` ізо скріншотами до і після. Збережіть також самі трейси (`.json` з Performance), вони знадобляться на співбесіді.
- Скріншот або лог, який показує, що regression-тест падає на початковому коді.

Звіт можна писати з допомогою будь-яких інструментів, але цифри й трейси мають бути вашими. На співбесіді ви відтворюєте їх наживо.

Додаткові умови (за бажанням, без балів)

1. Додати до застосунку бібліотеку `web-vitals` і виводити LCP, INP та CLS у консоль. Це дасть порівняння з Lighthouse.

2. Провести аудит на слабшому пристрої або телефоні через remote debugging і порівняти з ноутбуком.

Це може стати темою розмови на співбесіді, але на оцінку не впливає.

Критерії оцінювання

Максимум — 12 балів. Оцінюється вміння виміряти й пояснити, а не обсяг звіту.

Частина	Що оцінюється	Бали
Виправлення з доказами	Базова лінія й методика: prod-збірка, умови, медіана з ⩾ 3 запусків	1
	Кожне підтверджене виправлення справжнього вузького місця (цифри + трейс + причина)	по 1, до 4
Regression-тест	Тест перевіряє поведінку, яка була зламана	1
	Доведено, що тест падає на початковому коді й проходить після виправлення	1
Співбесіда	Знімає трейс наживо й показує в ньому проблему	2
	Пояснює причину й чому виправлення працює	2
	Відповідає на додаткові питання	1
Разом		**12**

Правила:

- Зміни, які не дали вимірюваного ефекту, балів не додають. Оптимізація «на всяк випадок» — це не виправлення.
- Якщо на співбесіді ви не можете показати в трейсі й пояснити своє виправлення, бал за нього не зараховується.
- Зламана функціональність або тести, що падають, — мінус 1 бал за кожен випадок.
- Абсолютні цифри на різних ноутбуках різняться. Оцінюється відносне покращення й коректна методика, а не збіг з чимось чужим результатом.

Контрольні запитання

1. Що таке Core Web Vitals? Які метрики до них входять і які їхні пороги?
2. Чому метрики оцінюють за 75-м перцентилем, а не за середнім?
3. З яких фаз складається LCP?
4. Що вимірює INP і з яких частин складається затримка взаємодії?
5. Як рахується CLS і в чому він вимірюється?
6. Що таке long task і чому межа саме 50 мс?
7. Чим відрізняються TBT і INP? Чому Lighthouse у режимі Navigation не показує INP?
8. Чим відрізняються lab data та field data?
9. Навіщо вимірювати на production-збірці й з throttling?
10. Чому один запуск Lighthouse не є доказом? Що таке шум вимірювань?
11. Як читати flame chart у панелі Performance? Чим відрізняються вкладки Call Tree і Bottom-Up?
12. Що показує React Profiler, чого не покаже панель Performance, і навпаки?
13. Чому main thread у браузері один? Як довга задача впливає на чуйність сторінки?
14. Який порядок дій при оптимізації: базова лінія, гіпотеза, зміна, повторне вимірювання? Чому міняти треба по одній речі за раз?
15. Чому код, що «виглядає повільним», не обов'язково є вузьким місцем?